

JDBC (Continue)

Scrollable and Updatable Result Sets

Overview

You know, in JDBC, a statement that executes a SQL SELECT query returns a result set represented by the `java.sql.ResultSet` object. By default, you can iterate over rows in the result set in forward direction only, from the first row to the last row, and you can't update data in the result set.

What if you need to **move forth and back in the result set**, jumping on any row when needed? Also, wouldn't it be more naturally to retrieve a result set and update its data against the database?

The JDBC API allows you to use such flexible result sets by making them *scrollable* and *updatable*.

1. Scrollable Result Sets

The single most visible addition to the JDBC API in its 2.0 specification is support for scrollable result sets.

Scrollable result sets are common in database vendor APIs, and the database vendors thus believed they should be present in JDBC.

Result Set Types

Using scrollable result sets starts with the way in which you create statements. Earlier in the chapter, you learned to create a statement using the `createStatement()` method. The `Connection` class actually has two versions of `createStatement()`—the zero parameter version you have used so far and a two parameter version that supports the creation of `Statement` instances that generate scrollable `ResultSet` objects. The default call translates to the following call:

```
conn.createStatement(ResultSet.TYPE_FORWARD_ONLY,  
                    ResultSet.CONCUR_READ_ONLY);
```

Use the following methods to scroll through the Scrollable result set:

- `first()`: moves the cursor to the first row.
 - `next()`: moves the cursor forward one row from its current position.
 - `previous()`: moves the cursor to the previous row.
-

2. Updatable Result Sets

As we know that one of the parameters used to create a scrollable result set is something called the *result set concurrency*. So far, the statements we have used the default concurrency, `ResultSet.CONCUR_READ_ONLY`. In other words, you cannot make changes to data in the result sets you have seen without creating a new update statement based on the data from your result set. Along with scrollable result sets, JDBC 2.0 also introduces the concept of *updatable result sets*—result sets you can change.

An updatable result set enables you to perform in-place changes to a result set and have them take effect using the current transaction. I place this discussion after batch processing because the only place it really makes sense in an enterprise environment is in large-scale batch processing. An overnight interest-assignment process for a bank is an example of such a potential batch process. It would read in an accounts balance and interest rate and, while positioned at that row in the database, update the interest. You naturally gain efficiency in processing since you do everything at once. The downside is that you perform database access and business logic together.

JDBC 2.0 result sets have two types of concurrency: `ResultSet.CONCUR_READ_ONLY` and `ResultSet.CONCUR_UPDATABLE`. You already know how to create an updatable result set from the discussion of scrollable result sets in previous section. You pass the concurrency type `ResultSet.CONCUR_UPDATABLE` as the second argument to `createStatement()`, or the third argument to `prepareStatement()` or `prepareCall()`:

```
PreparedStatement stmt = conn.prepareStatement(
    "SELECT acct_id, balance FROM account",
    ResultSet.TYPE_SCROLL_SENSITIVE,
    ResultSet.CONCUR_UPDATABLE);
```

The most important thing to remember about updatable result sets is that you must always select from a single table and include the primary key columns. If you don't,

the concept of the result set being updatable is nonsensical. After all, updatable result set only constructs a hidden `UPDATE` for you. If it does not know what the unique identifier for the row in question is, there is no way it can construct a valid update.

2.1 Updates

JDBC 2.0 introduces a set of `updateXXX()` methods to match its `getXXX()` methods and enable you to update a result set. For example, `updateString(1, "violet")` enables your application to replace the current value for column 1 of the current row in the result set with a `String` that has the value `violet`. Once you are done modifying columns, call `updateRow()` to make the changes permanent in the database. You naturally cannot make changes to primary key columns. Updates look like this:

```
while( rs.next( ) ) {
    long acct_id = rs.getLong(1);
    double balance = rs.getDouble(2);

    balance = balance + (balance * 0.03)/12;
    rs.updateDouble(2, balance);
    rs.updateRow( );
}
```

2.2 Deletes

Deletes are naturally much simpler than updates. Rather than setting values, you just have to call `deleteRow()`. This method will delete the current row out from under you and out of the database.

2.3 Inserts

Inserting a new row into a result set is the most complex operation of updatable result sets because inserts introduce a few new steps into the process. The first step is to create a row for update via the method `moveToInsertRow()`. This method creates a row that is basically a scratchpad for you to work in. This new row becomes your current row. As with other rows, you can call `getXXX()` and `updateXXX()` methods on it to retrieve or modify its values. Once you are done making changes, call `insertRow()` to make the changes permanent.

Any values you fail to set are assumed to be `null`. The following code demonstrates the insertion of a new row using an updatable result set:

```
rs.moveToInsertRow( );  
rs.updateString(1, "newuid");  
rs.updateString(2, "newpass");  
rs.insertRow( );  
rs.moveToCurrentRow( );
```

The seemingly peculiar call to `moveToCurrentRow( )` returns you to the row you were on before you attempted to insert the new row.